

TP 3 - Factorisation / Views

<https://docs.maximelefevre.fr/docs/cours/pythonflask>

Partie 1 - Setup

Avec Flask pour appeler des images, fichier CSS ou autre fichier voici comment vous

devez vous y prendre :

<!-- Il s'agit ici d'exemple -->

<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">

Vous êtes libre de choisir une autre structure.

Après avoir mis en place votre structure, créer votre server python Flask, pour cela

utiliser les anciens TP's et cours.

Cette partie sera terminée lorsque vous aurez démarré votre serveur Flask en mode

Développement.

2. À l'intérieur de ce dossier, créez la structure suivante :

```
mon_portfolio/
├── app.py
├── static/
│   ├── css/
│   │   └── style.css
│   ├── images/
│   ├── files/
│   │   └── cv.pdf
│   └── templates/
│       ├── layouts/
│       │   ├── base.html
│       │   ├── header.html
│       │   ├── footer.html
│       │   ├── competences.html
│       │   ├── formations.html
│       │   └── contact.html
│       └── index.html
└── requirements.txt
```

Les commande

```
virtual environment instead: https://pip.pypa.io/warnings/venv
(venv) administrateur@rt-mv:~/mon_portfolio$ touch app.py requirements.txt
(venv) administrateur@rt-mv:~/mon_portfolio$ mkdir -p static/css static/images s
static/files
(venv) administrateur@rt-mv:~/mon_portfolio$ mkdir -p templates/layouts
(venv) administrateur@rt-mv:~/mon_portfolio$ touch static/css/style.css
(venv) administrateur@rt-mv:~/mon_portfolio$ touch static/files/cv.pdf
(venv) administrateur@rt-mv:~/mon_portfolio$ touch templates/index.html
(venv) administrateur@rt-mv:~/mon_portfolio$ touch templates/layouts/{base.html,
header.html,footer.html,competences.html,formations.html,contact.html}
(venv) administrateur@rt-mv:~/mon_portfolio$ tree mon_portfolio
mon_portfolio
├── app.py
├── requirements.txt
├── static
│   ├── css
│   │   └── style.css
│   ├── files
│   │   └── cv.pdf
│   └── images
├── templates
│   ├── {index.html}
│   └── layouts
│       ├── base.html
│       ├── competences.html
│       ├── contact.html
│       ├── footer.html
│       ├── formations.html
│       └── header.html
└── 6 directories, 11 files
(venv) administrateur@rt-mv:~/mon_portfolio$
```

1. Objectif du TP

L'objectif de ce TP est de développer une application web avec Flask en respectant les consignes suivantes :

- Coder un script serveur Flask
- Ajouter des éléments en HTML (liste, lien, formulaire)
- Gérer les routes et le rendu des pages
- Assurer la propreté du code et des commentaires
- Améliorer le visuel du site
- Implémenter une fonctionnalité bonus avec JavaScript

Déroulement des tâches

- 2.1. Mise en place du serveur Flask

Le script de démarrage du serveur est défini dans un fichier `app.py` avec Flask.

- `# Importation des modules nécessaires`
- `from flask import Flask, render_template`
-
- `# Création de l'application Flask`
- `app = Flask(__name__)`
-
- `# Route principale du site`
- `@app.route('/')`
- `def home():`
- `return render_template('index.html')`
-
- `# Route pour la page de contact`
- `@app.route('/contact')`
- `def contact():`
- `return render_template('contact.html')`
-
- `# Lancement du serveur sur le port 5000\ nif __name__ == '__main__':`
- `app.run(port=5000, debug=True)`

Add this app.py

Puis make sure to add the programme in index.html where u can affiche some on the page

Make sure u are in right La repertoire de fichier :

```
cd /home/administrateur/abcd
```

Puis

```
python3 app.py
```

DONC LA SERVEUSE DOIS MARCHE SUR LA PORTE 500

- 2.2. Ajout d'une liste HTML (Ajouter une liste en HTML contenant au moins 3 éléments)

Dans `index.html`, j'ai ajouté une liste avec 3 éléments

- `<ul>`
 - `<li>Accueil</li>`
 - `<li>Services</li>`
 - `<li>Contact</li>`
- `</ul>`

- 1.3. Ajout d'un lien cliquable vers un nouvel onglet (Ajouter un lien cliquable vers un nouvel onglet)

Dans cette partie l'objectif est d'insérer un lien HTML qui, lorsqu'on clique dessus, ouvre une nouvelle page dans un nouvel onglet du navigateur.

J'ai ajouté un lien qui s'ouvre dans un nouvel onglet avec `target="_blank"`

```
<a href="https://fishy-poppy-d12.notion.site/Compte-rendu-153b1af10aa5801ca607ee16394246ec?pvs=4" target="_blank">Visitez notre site partenaire</a>
```

- 2.4. Création d'une page "contact.html" avec un formulaire

- `<form action="#" method="post" onsubmit="confirmSubmission(event)">`
- `<label for="name">Nom complet :</label>`
- `<input type="text" id="name" name="name" required>`
-
- `<label for="email">Adresse email :</label>`
- `<input type="email" id="email" name="email" required>`
-
- `<label for="subject">Sujet :</label>`
- `<select id="subject" name="subject">`
- `<option value="support">Support</option>`
- `<option value="info">Informations</option>`
- `<option value="autre">Autre</option>`
- `</select>`
-
- `<label for="message">Message :</label>`
- `<textarea id="message" name="message" required></textarea>`

```

-
-     <button type="submit">Envoyer</button>
- </form>
-
- <script>
- function confirmSubmission(event) {
-     event.preventDefault(); // Empêche l'envoi par défaut
-     alert("Votre message a bien été envoyé !");
- }
- </script>
-

```

Ajout de la route dans `app.py`

```

• @app.route('/contact')
• def contact():
•     return render_template('contact.html')

```

3. Notation complémentaire

3.1. Commentaires dans le code

Je vais ajouter des commentaires pour expliquer le code :
Exemple dans `app.py`

```

• # Importation des modules nécessaires
• from flask import Flask, render_template
•
• # Création de l'application Flask
• app = Flask(__name__)
•
• # Route principale du site
• @app.route('/')
• def home():
•     ◦ return render_template('index.html')
•
• # Route pour la page de contact
• @app.route('/contact')
• def contact():
•     ◦ return render_template('contact.html')
•
• # Lancement du serveur sur le port 5000
• if __name__ == '__main__':
•     ◦ app.run(port=5000, debug=True)

```

- 3.3. Visuel du site (3 points)

J'ai ajouté du CSS pour améliorer le design :

- body {
- font-family: Arial, sans-serif;
- background-color: #f4f4f4;
- padding: 20px;
- }
-
- form {
- background: white;
- padding: 20px;
- border-radius: 5px;
- box-shadow: 0px 0px 10px #ccc;
- width: 50%;
- margin: auto;
- }
-
- button {
- background-color: #008CBA;
- color: white;
- border: none;
- padding: 10px;
- cursor: pointer;
- }
-
- button:hover {
- background-color: #005f6b;
- }

4. JavaScript dynamique

Je vais ajouter un script pour afficher le nom entré en temps réel

```
<h2 id="display-name">Votre nom : </h2>
```

```
<input type="text" id="name" name="name" oninput="updateName()">
```

```
<script>
```

```
function updateName() {
```

```
    let name = document.getElementById("name").value;
```

```
    document.getElementById("display-name").innerText = "Votre nom : " + name;
```

```
}
```

```
</script>
```